

AN2DL - First Challenge Report

Reluminati

Serkan Basaran, Margarita Makurina, Karim Negm, Muhammet Emre Eren

SerkanBasaran1, margaritamakurina, karimynegm, muhammet emre eren

276107, 270455, 273117, 277693

November 17, 2025

1 Introduction

The goal of this project is to develop a **time series sequence** model for the *Pirate Pain Prediction* task, classifying a pirate's **pain level** (high, low or absence) based on multivariate physiological, behavioral, and categorical temporal sequences. Unlike static classification, this requires the model to infer class-dependent dynamics from temporal patterns, making architectures that capture sequential dependencies, such as **Recurrent Neural Networks (RNNs)** [1] and their gated variants, particularly suitable.

2 Problem Analysis

The Pirate Pain data set is a multivariate time series classification task. Each pirate (identified by `sample_index`) is recorded for approximately 160 consecutive timesteps.

2.1 Feature Groups

The 40 features fall into four categories: **Identification & Time** (`sample_index`, `time`, `t=0-159`); **Static Categorical** (`n_legs`, `n_hands`, `n_eyes`); **Temporal Categorical** (four `pain_survey` features at each timestep); **Temporal Continuous** (31 `joint_signals`).

2.2 Main Challenges

The dataset presents three primary challenges. First, it suffers from **highly imbalanced classes**. The `no_pain` class dominates the dataset. Second, it contains **long and variable sequences** in which pain-indicating patterns may be short, transient, and occur anywhere within the 160-timestep sequence. Finally, it includes **mixed data types**, requiring the model to handle continuous sensor data, temporal categorical data, and static categorical data simultaneously.

2.3 Implications for Model Design

These challenges directly inform the model design. A recurrent architecture, such as **GRU or LSTM** [2], is required to learn dependencies within the temporal sequences. The class imbalance necessitates a specialized loss function, such as **Class-Weighted Cross-Entropy**, to ensure minority classes are properly learned. Additionally, the mixed-data types require a hybrid architecture that uses **Embedding layers** for categorical features before combining them with the continuous sensor data.

3 Method

Our method employs a hybrid **Conv1D-GRU** architecture, using **embedding** layers to process cate-

gorical features and a **class-weighted loss** to handle data imbalance. Hyper parameters are tuned by **Optuna**[3]. Final predictions are generated by averaging probability outputs from a **K-Fold ensemble** [4].

3.1 Data Preprocessing

Data Analysis:

- The `joint_30` feature had **zero variance** across all samples and timesteps.
- `joint_10` and `joint_11` were found to be 95% correlated.
- The `num_eyes`, `num_legs`, and `num_hands` features were identical per sample, so these were replaced with a single binary feature called `is_pirate`.

Feature Scaling:

- To **normalize** the model’s inputs, a **StandardScaler** [5] was fitted to the training data and applied to all 30 continuous `joint_xx` features.

Windowing:

- The 160-step sequences were transformed into overlapping windows. A **window size of 10 and stride of 2** were selected based on the autocorrelation function (ACF) of the data, which revealed very short memory characteristics. These parameters were also consistently chosen by **hyperparameter optimization (HPO)** across the majority of trials.

Hold-Out Set:

- A 10% **hold-out test set** was created at the beginning of the workflow. This set was reserved and used only for the final, unbiased evaluation of the **K-Fold ensemble**.

3.2 Model Architecture

Input windows first pass through the following layers in sequential order:

- **Embedding layers** [6] convert categorical features (e.g., `pain_survey`) into dense vectors.
- In the **concatenation** step, the embedding vectors are combined with the continuous `joint_` features at each timestep.
- The **Conv1D layer** [7] then acts as a feature extractor, identifying local patterns in the combined sequence.
- The **Bidirectional GRU layer** [2] captures long-range temporal dependencies and context from both past and future timesteps.
- **Dropout layers** [8] are applied for regularization to prevent overfitting.
- A **Linear classifier** [9], which is a fully-connected layer, outputs the 3 class logits for the given window.

3.3 Training Strategy and Optimization

Loss Function: For the loss function **Class-Weighted Cross-Entropy** [10] (formula 1) was used to address class imbalance by penalizing errors on minority classes.

$$\mathcal{L} = - \sum_{i=1}^N w_i y_i \log(p_i) \quad (1)$$

Learning Rate Scheduler: The learning rate scheduler employed **CosineAnnealingLR** [11, 12] to adjust the learning rate, promoting faster convergence.

Hyperparameter Optimization: For hyperparameter optimization, **Optuna** [3] was used to efficiently tune key model and training parameters (e.g., learning rate, layer size, dropout).

Final Prediction: Finally, a **K-Fold ensemble** [4] was used for inference. Final predictions were derived by averaging the probabilities from all overlapping windows across all folds (formula 2).

$$\hat{\mathbf{p}} = \frac{1}{K} \sum_{k=1}^K \left(\frac{1}{W} \sum_{j=1}^W \mathbf{p}_{k,j} \right) \quad (2)$$

Table 1: Model Performance Comparison by Ensemble Strategy. Best results are highlighted in **bold**.

Model	Ensemble Strategy	Validation F1 (Mean $\pm$ Std)	Kaggle F1 Score
Hold-Out Test Ensemble	5-Fold Average (90% Train Data)	93.71 $\pm$ 0.75	94.93
No Test Split Ensemble	5-Fold Average (100% Train Data)	93.56 $\pm$ 2.12	95.70
No Test Split + Fold Pruning	3-Fold Pruned Average	95.16 $\pm$ 1.02	96.04

4 Experiments

Our experimental process had three stages, summarized in Table 1. We first trained a **Hold-Out Ensemble** (90% data) for a baseline, then a **No Test Split Ensemble** (100% data). Our final **Pruned Ensemble**, which averaged only the best-performing folds, achieved our highest competition score.

5 Results

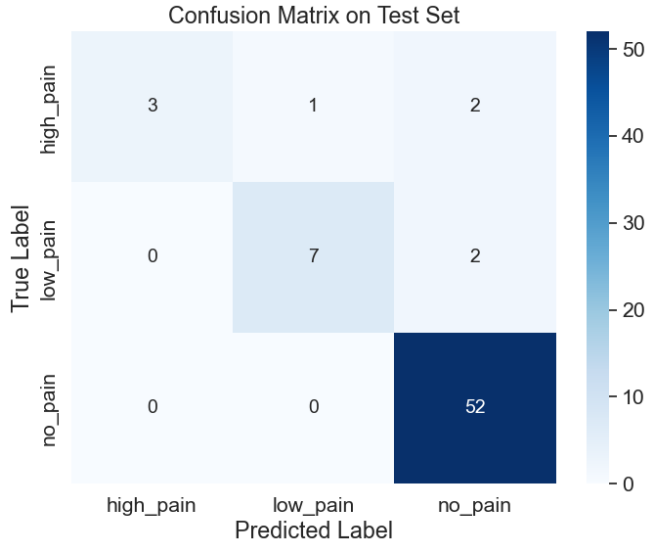


Figure 1: Test Set Confusion Matrix

- The **no_pain** class achieved an F1-Score of 0.96, with 0.93 Precision and 1.0 Recall.
- The **low_pain** class obtained an F1-Score of 0.82, with 0.88 Precision and 0.78 Recall.
- The **high_pain** class had an F1-Score of 0.67, with 1.0 Precision and 0.50 Recall.
- Overall, the test set achieved a weighted **F1-Score of 0.92**.

6 Discussion

- The data suggests there is room for improvement in class balancing, as performance degrades for rarer classes. Techniques such as focal loss functions or oversampling are worth exploring to address this issue.
- The method by which we scale the data is likely also a contributing factor, given the surprisingly high precision but poor recall on the **high_pain** class.
- While pruning out the underperforming folds significantly improved the Kaggle scores, this approach may be overfitting to the public test set and could result in decreased performance on the hidden dataset.
- Given the small test sample size of 67 samples, these findings must be interpreted with caution.

7 Conclusions

We developed a deep learning model that combines convolutional and recurrent layers (Conv1D-GRU) to classify pirate pain from time-series sensor and survey data. Our best result, a **96.04 Kaggle F1-score**, was achieved by averaging predictions from our most stable models after training them on all available data. While the model is highly accurate, it struggles to identify the rarest pain category, indicating that our class-weighting method was insufficient and future work should focus on better balancing techniques. Additionally, an Attention mechanism could be added after the GRU layer to learn a weighted sum of timesteps, allowing the model to focus on critical pain spikes instead of just the final state.

References

- [1] Matteo Matteucci. Recurrent neural networks. lecture notes, course: Artificial Neural Networks and Deep Learning, 2025. Email: matteo.matteucci@polimi.it.
- [2] Pytorch: Gru layer. <https://docs.pytorch.org/docs/stable/generated/torch.nn.GRU.html>. Accessed: 2025-11-10.
- [3] Optuna: A hyperparameter optimization framework. <https://optuna.org/>. Accessed: 2025-11-16.
- [4] Scikit-learn: Kfold cross-validation. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.KFold.html. Accessed: 2025-11-15.
- [5] Scikit-learn: StandardScaler. <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>. Accessed: 2025-11-14.
- [6] Pytorch: Embedding layer. <https://docs.pytorch.org/docs/stable/generated/torch.nn.Embedding.html>. Accessed: 2025-11-08.
- [7] Pytorch: Conv1d layer. <https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv1d.html>. Accessed: 2025-11-08.
- [8] Pytorch: Dropout. <https://docs.pytorch.org/docs/stable/generated/torch.nn.Dropout.html>. Accessed: 2025-11-09.
- [9] Pytorch: Linear layer. <https://docs.pytorch.org/docs/stable/generated/torch.nn.Linear.html>. Accessed: 2025-11-09.
- [10] Pytorch: Crossentropyloss. <https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html>. Accessed: 2025-11-11.
- [11] Pytorch: Cosineannealinglr. https://docs.pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.CosineAnnealingLR.html. Accessed: 2025-11-11.
- [12] Utkrisht Mallick. Cosine learning rate schedulers in pytorch. Medium, October 2024. Accessed: 2025-11-17.